Classification using sklearn and keras (with pandas)

File access required: In Colab this notebook requires first uploading files **Cities.csv**, **Players.csv**, and **Titanic.csv** using the *Files* feature in the left toolbar. If running the notebook on a local computer, simply ensure these files are in the same workspace as the notebook.

```
# Set-up
import csv
import numpy as np
import pandas as pd
from sklearn.neighbors import KNeighborsClassifier
from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import RandomForestClassifier
from sklearn.naive_bayes import GaussianNB
from sklearn.preprocessing import LabelEncoder
from sklearn.preprocessing import StandardScaler
from keras import Sequential
from keras.layers import Dense
from numpy.random import seed
import tensorflow
```

Prepare Cities data for classification

Predict temperature category from other features

```
# Read Cities.csv into dataframe, add column for temperature category
# Note: For a dataframe D and integer i, D.loc[i] is the i-th row of D
f = open('Cities.csv')
cities = pd.read_csv(f)
categories = []
for i in range(len(cities)):
    if cities.loc[i]['temperature'] < 5:
        categories.append('cold')
    elif cities.loc[i]['temperature'] < 9:
        categories.append('cool')
    elif cities.loc[i]['temperature'] < 15:
        categories.append('warm')
    else: categories.append('hot')
cities['category'] = categories
print("cold:", len(cities[(cities.category == 'cold')]))
print("cool:", len(cities[(cities.category == 'cool')]))
print("warm:", len(cities[(cities.category == 'warm')]))
print("hot:", len(cities[(cities.category == 'hot')]))
```

```
cold: 17
cool: 92
warm: 79
hot: 25
```

```
# Create training and test sets for cities data
numitems = len(cities)
percenttrain = 0.85
numtrain = int(numitems*percenttrain)

print('Training set', numtrain, 'items')
print('Test set', numitems - numtrain, 'items')

citiesTrain = cities[0:numtrain]
citiesTest = cities[numtrain:]
```

```
Training set 181 items
Test set 32 items
```

K-nearest-neighbors classification

```
features = ['longitude', 'latitude']
neighbors = 3
predict = 'category'

classifier = KNeighborsClassifier(neighbors)
classifier.fit(citiesTrain[features], citiesTrain[predict])


predictions = classifier.predict(citiesTest[features])

# Calculate accuracy
actuals = list(citiesTest[predict])
correct = 0

for i in range(len(actuals)):
  print('Predicted:', predictions[i], ' Actual:', actuals[i])
  if predictions[i] == actuals[i]: correct +=1
print('Accuracy:', round(correct/len(actuals),5))
# Comment out print, try different values for neighbors, different
features
```

```
Predicted: warm  Actual: cool
Predicted: warm  Actual: warm
Predicted: hot  Actual: warm
Predicted: warm  Actual: warm
Predicted: cold  Actual: cool
Predicted: cool  Actual: cool
Predicted: cool  Actual: cool
Predicted: warm  Actual: warm
Predicted: warm  Actual: warm
Predicted: cool  Actual: cold
Predicted: cold  Actual: cold
Predicted: cool  Actual: warm
Predicted: cool  Actual: cold
```

```
Predicted: warm  Actual: warm
Predicted: warm  Actual: warm
Predicted: cool  Actual: warm
Predicted: warm  Actual: warm
Predicted: hot  Actual: hot
Predicted: cold  Actual: cold
Predicted: cold  Actual: cold
Predicted: cool  Actual: cold
Predicted: hot  Actual: hot
Predicted: warm  Actual: cool
Predicted: warm  Actual: warm
Predicted: cool  Actual: cool
Predicted: cool  Actual: cool
Predicted: cool  Actual: cool
Predicted: cool  Actual: warm
Predicted: warm  Actual: warm
Predicted: cool  Actual: cool
Predicted: warm  Actual: warm
Predicted: cool  Actual: cool
Accuracy: 0.6875
```

Your Turn: K-nearest-neighbors on World Cup data

Predict position from one or more of minutes, shots, passes, tackles, saves

```
# This cell does all the set-up, including reordering the data to
avoid team bias.
f = open('Players.csv')
players = pd.read_csv(f)
players = players.sort_values(by='surname')
players = players.reset_index(drop=True)
numitems = len(players)
percenttrain = 0.92
numtrain = int(numitems*percenttrain)
print('Training set', numtrain, 'items')
print('Test set', numitems - numtrain, 'items')
playersTrain = players[0:numtrain]
playersTest = players[numtrain:]
```

```
Training set 547 items
Test set 48 items
```

```
# This cell does the classification.
# Try different features and different numbers of neighbors.
# What's the highest accuracy you can get?
features = ['shots', 'tackles']
neighbors = 25
predict = 'position'
classifier = KNeighborsClassifier(neighbors)
```

```
classifier.fit(playersTrain[features], playersTrain[predict])
predictions = classifier.predict(playersTest[features])
# Calculate accuracy
actuals = list(playersTest[predict])
correct = 0
for i in range(len(actuals)):
  print('Predicted:', predictions[i], ' Actual:', actuals[i])
  if predictions[i] == actuals[i]: correct +=1
print('Accuracy:', round(correct/len(actuals),5))
# Comment out print, try different values for neighbors, different
features
```

```
Predicted: defender  Actual: midfielder
Predicted: goalkeeper  Actual: goalkeeper
Predicted: forward  Actual: midfielder
Predicted: defender  Actual: defender
Predicted: midfielder  Actual: forward
Predicted: midfielder  Actual: midfielder
Predicted: midfielder  Actual: midfielder
Predicted: defender  Actual: midfielder
Predicted: defender  Actual: defender
Predicted: defender  Actual: defender
Predicted: midfielder  Actual: forward
Predicted: defender  Actual: defender
Predicted: defender  Actual: defender
Predicted: midfielder  Actual: defender
Predicted: forward  Actual: forward
Predicted: goalkeeper  Actual: goalkeeper
Predicted: defender  Actual: defender
Predicted: forward  Actual: forward
Predicted: defender  Actual: defender
Predicted: midfielder  Actual: forward
Predicted: forward  Actual: forward
Predicted: midfielder  Actual: midfielder
Predicted: forward  Actual: forward
Predicted: defender  Actual: midfielder
Predicted: defender  Actual: midfielder
Predicted: forward  Actual: forward
Predicted: midfielder  Actual: midfielder
Predicted: midfielder  Actual: midfielder
Predicted: defender  Actual: defender
Predicted: forward  Actual: midfielder
Predicted: forward  Actual: forward
Predicted: goalkeeper  Actual: forward
Predicted: midfielder  Actual: midfielder
Predicted: midfielder  Actual: midfielder
Predicted: defender  Actual: midfielder
Predicted: defender  Actual: defender
Predicted: defender  Actual: defender
Predicted: defender  Actual: defender
```

```
Predicted: midfielder  Actual: midfielder
Predicted: defender  Actual: defender
Predicted: forward  Actual: forward
Predicted: defender  Actual: midfielder
Predicted: defender  Actual: midfielder
Predicted: defender  Actual: midfielder
Predicted: midfielder  Actual: midfielder
Predicted: defender  Actual: defender
Predicted: forward  Actual: forward
Predicted: defender  Actual: defender
Accuracy: 0.6875
```

Your Turn Extra: K-nearest-neighbors on Titanic data - Graded

Predict survived from one or more of gender, age, class, fare, embarked

```
# This cell does all the set-up
f = open('Titanic.csv')
titanic = pd.read_csv(f)
# Convert gender and embarked to numeric values and missing ages to
average age
titanic['gender'].replace({'M':0, 'F':1}, inplace=True)
titanic['embarked'].replace({'Cherbourg':0, 'Southampton':1,
'Queenstown':2}, inplace=True)
avg_age = np.average(titanic['age'].dropna().tolist())
titanic['age'].fillna(avg_age, inplace=True)
# Create training and test sets
numitems = len(titanic)
percenttrain = 0.92
numtrain = int(numitems*percenttrain)
print('Training set', numtrain, 'items')
print('Test set', numitems - numtrain, 'items')
titanicTrain = titanic[0:numtrain]
titanicTest = titanic[numtrain:]
```

```
Training set 819 items
Test set 72 items
```

```
/tmp/ipython-input-257007092.py:5: FutureWarning: A value is trying to
be set on a copy of a DataFrame or Series through chained assignment
using an inplace method.
The behavior will change in pandas 3.0. This inplace method will never
work because the intermediate object on which we are setting values
always behaves as a copy.

For example, when doing 'df[col].method(value, inplace=True)', try
using 'df.method({col: value}, inplace=True)' or df[col] =
df[col].method(value) instead, to perform the operation inplace on the
original object.
```

```
  titanic['gender'].replace({'M':0, 'F':1}, inplace=True)
/tmp/ipython-input-257007092.py:5: FutureWarning: Downcasting behavior
in `replace` is deprecated and will be removed in a future version. To
retain the old behavior, explicitly call
`result.infer_objects(copy=False)`. To opt-in to the future behavior,
set `pd.set_option('future.no_silent_downcasting', True)`
  titanic['gender'].replace({'M':0, 'F':1}, inplace=True)
/tmp/ipython-input-257007092.py:6: FutureWarning: A value is trying to
be set on a copy of a DataFrame or Series through chained assignment
using an inplace method.
The behavior will change in pandas 3.0. This inplace method will never
work because the intermediate object on which we are setting values
always behaves as a copy.

For example, when doing 'df[col].method(value, inplace=True)', try
using 'df.method({col: value}, inplace=True)' or df[col] =
df[col].method(value) instead, to perform the operation inplace on the
original object.


  titanic['embarked'].replace({'Cherbourg':0, 'Southampton':1,
'Queenstown':2}, inplace=True)
/tmp/ipython-input-257007092.py:6: FutureWarning: Downcasting behavior
in `replace` is deprecated and will be removed in a future version. To
retain the old behavior, explicitly call
`result.infer_objects(copy=False)`. To opt-in to the future behavior,
set `pd.set_option('future.no_silent_downcasting', True)`
  titanic['embarked'].replace({'Cherbourg':0, 'Southampton':1,
'Queenstown':2}, inplace=True)
/tmp/ipython-input-257007092.py:8: FutureWarning: A value is trying to
be set on a copy of a DataFrame or Series through chained assignment
using an inplace method.
The behavior will change in pandas 3.0. This inplace method will never
work because the intermediate object on which we are setting values
always behaves as a copy.

For example, when doing 'df[col].method(value, inplace=True)', try
using 'df.method({col: value}, inplace=True)' or df[col] =
df[col].method(value) instead, to perform the operation inplace on the
original object.


  titanic['age'].fillna(avg_age, inplace=True)
```

```
# This cell does the classification.
# Try different features and different numbers of neighbors.
# What's the highest accuracy you can get?
features = ['gender', 'age', 'class', 'fare', 'embarked']
```

```
neighbors = 10
predict = 'survived'
classifier = KNeighborsClassifier(neighbors)
classifier.fit(titanicTrain[features], titanicTrain[predict])
predictions = classifier.predict(titanicTest[features])
# Calculate accuracy
actuals = list(titanicTest[predict])
correct = 0
for i in range(len(actuals)):
# print('Predicted:', predictions[i], ' Actual:', actuals[i])
  if predictions[i] == actuals[i]: correct +=1
print('Accuracy:', round(correct/len(actuals),5))
```

```
Accuracy: 0.73611
```

Decision tree classification

```
features = ['latitude', 'longitude']
split = 2
predict = 'category'

# random forest
for x in range(1, 10):
  dt = DecisionTreeClassifier(random_state=0, min_samples_split=split)
# split parameter is optional
  dt.fit(citiesTrain[features], citiesTrain[predict])

  predictions = dt.predict(citiesTest[features])
  # print(x ....)

# aggregated predicted output



# Calculate accuracy
actuals = list(citiesTest[predict])
correct = 0
for i in range(len(actuals)):
# print('Predicted:', predictions[i], ' Actual:', actuals[i])
  if predictions[i] == actuals[i]: correct +=1
print('Accuracy:', round(correct/len(actuals),5))
# Try different values for split, different features
```

```
Accuracy: 0.75
```

"Forest" of decision trees

```
features = ['longitude', 'latitude']
split = 10
trees = 10
```

```
predict = 'category'

rf = RandomForestClassifier(random_state=0, min_samples_split=split,
n_estimators=trees)
rf.fit(citiesTrain[features], citiesTrain[predict])


predictions = rf.predict(citiesTest[features])
# Calculate accuracy
actuals = list(citiesTest[predict])
correct = 0
for i in range(len(actuals)):
# print('Predicted:', predictions[i], ' Actual:', actuals[i])
  if predictions[i] == actuals[i]: correct +=1
print('Accuracy:', round(correct/len(actuals),5))
# Try different values for split and trees, different features
```

```
Accuracy: 0.78125
```

Your Turn: Decision tree and forest of trees on World Cup data - Graded

```
# SINGLE TREE
# Try different features and different values for split.
# What's the highest accuracy you can get?
features = ['minutes', 'shots', 'passes', 'tackles', 'saves']
split = 34
predict = 'position'
dt = DecisionTreeClassifier(random_state=0, min_samples_split=split) #
parameter is optional
dt.fit(playersTrain[features], playersTrain[predict])
predictions = dt.predict(playersTest[features])
# Calculate accuracy
actuals = list(playersTest[predict])
correct = 0
for i in range(len(actuals)):
# print('Predicted:', predictions[i], ' Actual:', actuals[i])
  if predictions[i] == actuals[i]: correct +=1
print('Accuracy:', round(correct/len(actuals),5))
```

```
Accuracy: 0.70833
```

```
# FOREST OF TREES
# Try different features and different values for split and trees.
# What's the highest accuracy you can get?
features = ['minutes', 'shots', 'passes', 'tackles', 'saves']
split = 25
trees = 100
predict = 'position'
rf = RandomForestClassifier(random_state=0, min_samples_split=split,
```

```
n_estimators=trees)
rf.fit(playersTrain[features], playersTrain[predict])
predictions = rf.predict(playersTest[features])
# Calculate accuracy
actuals = list(playersTest[predict])
correct = 0
for i in range(len(actuals)):
# print('Predicted:', predictions[i], ' Actual:', actuals[i])
  if predictions[i] == actuals[i]: correct +=1
print('Accuracy:', round(correct/len(actuals),5))
```

```
Accuracy: 0.77083
```

Your Turn Extra: Decision tree and forest of trees on Titanic data - Graded

```
# SINGLE TREE
# Try different features and different values for split.
# What's the highest accuracy you can get?
features = ['gender', 'class']
split = 2
predict = 'survived'
dt = DecisionTreeClassifier(random_state=0, min_samples_split=split) #
parameter is optional
dt.fit(titanicTrain[features], titanicTrain[predict])
predictions = dt.predict(titanicTest[features])
# Calculate accuracy
actuals = list(titanicTest[predict])
correct = 0
for i in range(len(actuals)):
# print('Predicted:', predictions[i], ' Actual:', actuals[i])
  if predictions[i] == actuals[i]: correct +=1
print('Accuracy:', round(correct/len(actuals),5))
```

```
Accuracy: 0.81944
```

```
# FOREST OF TREES
# Try different features and different values for split and trees.
# What's the highest accuracy you can get?
features = ['gender', 'age', 'fare']
split = 2
trees = 5
predict = 'survived'
rf = RandomForestClassifier(random_state=0, min_samples_split=split,
n_estimators=trees)
rf.fit(titanicTrain[features], titanicTrain[predict])
predictions = rf.predict(titanicTest[features])
# Calculate accuracy
actuals = list(titanicTest[predict])
correct = 0
```

```
for i in range(len(actuals)):
# print('Predicted:', predictions[i], ' Actual:', actuals[i])
  if predictions[i] == actuals[i]: correct +=1
print('Accuracy:', round(correct/len(actuals),5))
```

```
Accuracy: 0.84722
```

Naive Bayes classification

```
features = [ 'longitude', 'latitude']
predict = 'category'

nb = GaussianNB()
nb.fit(citiesTrain[features], citiesTrain[predict])

predictions = nb.predict(citiesTest[features])

# Calculate accuracy
actuals = list(citiesTest[predict])
correct = 0
for i in range(len(actuals)):
# print('Predicted:', predictions[i], ' Actual:', actuals[i])
  if predictions[i] == actuals[i]: correct +=1
print('Accuracy:', round(correct/len(actuals),5))
# Try different features
```

```
Accuracy: 0.78125
```

Your Turn: Naive Bayes on World Cup data

```
# Try different features. What's the highest accuracy you can get?
features = ['shots', 'passes', 'tackles', 'saves']
predict = 'position'
nb = GaussianNB()
nb.fit(playersTrain[features], playersTrain[predict])
predictions = nb.predict(playersTest[features])
# Calculate accuracy
actuals = list(playersTest[predict])
correct = 0
for i in range(len(actuals)):
# print('Predicted:', predictions[i], ' Actual:', actuals[i])
  if predictions[i] == actuals[i]: correct +=1
print('Accuracy:', round(correct/len(actuals),5))
```

```
Accuracy: 0.75
```

Your Turn Extra: Naive Bayes on Titanic data - Graded

```
# Try different features. What's the highest accuracy you can get?
features = ['gender', 'age', 'class']
```

```
predict = 'survived'
nb = GaussianNB()
nb.fit(titanicTrain[features], titanicTrain[predict])
predictions = nb.predict(titanicTest[features])
# Calculate accuracy
actuals = list(titanicTest[predict])
correct = 0
for i in range(len(actuals)):
# print('Predicted:', predictions[i], ' Actual:', actuals[i])
  if predictions[i] == actuals[i]: correct +=1
print('Accuracy:', round(correct/len(actuals),5))
```

```
Accuracy: 0.81944
```

Neural network classification

```
features = ['longitude', 'latitude']
num_layers = 5 # including input and output, so must be >= 2
num_epochs = 10 # number of iterations over training data
batchsize = 20 # size of each batch during one iteration
layer_outputs = 32 # dimensionality of output of each layer
epoch_tracing = 'yes'
predict = 'category'
# Normalize feature values
sc = StandardScaler()
featurevals_train = sc.fit_transform(citiesTrain[features])
featurevals_test = sc.fit_transform(citiesTest[features])
# Encode labels
encoder = LabelEncoder()
encoder.fit(cities[predict])
labels_train = encoder.transform(citiesTrain[predict])
labels_test = encoder.transform(citiesTest[predict])
# Set up neural-net classifier
seed(1) # to eliminate some randomness
tensorflow.random.set_seed(1) # to eliminate more randomness
classifier = Sequential()
# Input layer
classifier.add(Dense(layer_outputs, activation='relu',
input_dim=len(features)))

# Hidden layers
for i in range(num_layers-2):
    classifier.add(Dense(layer_outputs, activation='relu',))


# Output layer - first arg is number of labels, softmax for multi-
class classification
classifier.add(Dense(4, activation='softmax'))
```

```
classifier.compile(optimizer ='adam',
loss='sparse_categorical_crossentropy', metrics =['accuracy'])

# Fit to training data
if epoch_tracing == 'yes': v = 2
else: v = 0
hist = classifier.fit(featurevals_train, labels_train,
batch_size=batchsize, epochs=num_epochs, verbose=v)
print('Number of epochs:', num_epochs)
print('Final accuracy on training data:', hist.history['accuracy'][-
1])
# Evaluate on test data
test_acc = classifier.evaluate(featurevals_test, labels_test,
verbose=0)[1]
print('Accuracy on test data:', test_acc)
# Try different values for num_layers, num_epochs, batch size,
layer_outputs, and different features
```

```
/usr/local/lib/python3.12/dist-packages/keras/src/layers/core/
dense.py:93: UserWarning: Do not pass an `input_shape`/`input_dim`
argument to a layer. When using Sequential models, prefer using an
`Input(shape)` object as the first layer in the model instead.
  super().__init__(activity_regularizer=activity_regularizer,
**kwargs)
```

```
Epoch 1/10
10/10 - 2s - 191ms/step - accuracy: 0.3978 - loss: 1.3637
Epoch 2/10
10/10 - 0s - 7ms/step - accuracy: 0.5359 - loss: 1.3068
Epoch 3/10
10/10 - 0s - 7ms/step - accuracy: 0.6354 - loss: 1.2481
Epoch 4/10
10/10 - 0s - 7ms/step - accuracy: 0.6851 - loss: 1.1775
Epoch 5/10
10/10 - 0s - 6ms/step - accuracy: 0.6796 - loss: 1.0936
Epoch 6/10
10/10 - 0s - 6ms/step - accuracy: 0.6796 - loss: 1.0057
Epoch 7/10
10/10 - 0s - 6ms/step - accuracy: 0.6851 - loss: 0.9272
Epoch 8/10
10/10 - 0s - 6ms/step - accuracy: 0.6796 - loss: 0.8674
Epoch 9/10
10/10 - 0s - 6ms/step - accuracy: 0.6851 - loss: 0.8198
Epoch 10/10
10/10 - 0s - 14ms/step - accuracy: 0.6851 - loss: 0.7794
Number of epochs: 10
Final accuracy on training data: 0.6850828528404236
Accuracy on test data: 0.625
```

Your Turn: Neural network on World Cup data

```
# Try different features and different values for num_layers,
num_epochs,
#  batch size, and layer_outputs.
# What's the highest accuracy you can get?
# Note: Although some randomness is removed by setting seeds in the
code,
#  you may still see somewhat different accuracy on different runs;
#  changing the order of the features can also affect accuracy
features = ['minutes', 'shots', 'tackles', 'saves']
num_layers = 4 # including input and output, so must be >= 2
num_epochs = 30 # number of iterations over training data
batchsize = 32 # size of each batch during one iteration
layer_outputs = 64 # dimensionality of output of each layer
epoch_tracing = 'no'
predict = 'position'
# Normalize feature values
sc = StandardScaler()
featurevals_train = sc.fit_transform(playersTrain[features])
featurevals_test = sc.fit_transform(playersTest[features])
# Encode labels
encoder = LabelEncoder()
encoder.fit(players[predict])
labels_train = encoder.transform(playersTrain[predict])
labels_test = encoder.transform(playersTest[predict])
# Set up neural-net classifier
seed(1) # to eliminate some randomness
tensorflow.random.set_seed(1) # to eliminate more randomness
classifier = Sequential()
# Input layer
classifier.add(Dense(layer_outputs, activation='relu',
input_dim=len(features)))
# Hidden layers
for i in range(num_layers-2):
    classifier.add(Dense(layer_outputs, activation='relu',))
# Output layer - first arg is number of labels, softmax for multi-
class classification
classifier.add(Dense(4, activation='softmax'))
classifier.compile(optimizer ='adam',
loss='sparse_categorical_crossentropy', metrics =['accuracy'])
# Fit to training data
if epoch_tracing == 'yes': v = 2
else: v = 0
hist = classifier.fit(featurevals_train, labels_train,
batch_size=batchsize, epochs=num_epochs, verbose=v)
print('Number of epochs:', num_epochs)
print('Final accuracy on training data:', hist.history['accuracy'][-
1])
# Evaluate on test data
```

```
test_acc = classifier.evaluate(featurevals_test, labels_test,
verbose=0)[1]
print('Accuracy on test data:', test_acc)
```

```
/usr/local/lib/python3.12/dist-packages/keras/src/layers/core/
dense.py:93: UserWarning: Do not pass an `input_shape`/`input_dim`
argument to a layer. When using Sequential models, prefer using an
`Input(shape)` object as the first layer in the model instead.
  super().__init__(activity_regularizer=activity_regularizer,
**kwargs)

Number of epochs: 30
Final accuracy on training data: 0.645338237285614
Accuracy on test data: 0.7291666865348816
```

Your Turn Extra: Neural network on Titanic data

```
# Try different features and different values for num_layers,
num_epochs,
#  batch size, and layer_outputs.
# What's the highest accuracy you can get?
# Note: Although some randomness is removed by setting seeds in the
code,
#  you may still see somewhat different accuracy on different runs;
#  changing the order of the features can also affect accuracy
features = ['gender', 'age', 'class', 'fare', 'embarked']
num_layers = 4 # including input and output, so must be >= 2
num_epochs = 20 # number of iterations over training data
batchsize = 32 # size of each batch during one iteration
layer_outputs = 64 # dimensionality of output of each layer
epoch_tracing = 'no'
predict = 'survived'
# Normalize feature values
sc = StandardScaler()
featurevals_train = sc.fit_transform(titanicTrain[features])
featurevals_test = sc.fit_transform(titanicTest[features])
# Encode labels
encoder = LabelEncoder()
encoder.fit(titanic[predict])
labels_train = encoder.transform(titanicTrain[predict])
labels_test = encoder.transform(titanicTest[predict])
# Set up neural-net classifier
seed(1) # to eliminate some randomness
tensorflow.random.set_seed(1) # to eliminate more randomness
classifier = Sequential()
# Input layer
classifier.add(Dense(layer_outputs, activation='relu',
input_dim=len(features)))
# Hidden layers
for i in range(num_layers-2):
```

```
    classifier.add(Dense(layer_outputs, activation='relu',))
# Output layer - first arg is number of labels, softmax for multi-
class classification
classifier.add(Dense(4, activation='softmax'))
classifier.compile(optimizer ='adam',
loss='sparse_categorical_crossentropy', metrics =['accuracy'])
# Fit to training data
if epoch_tracing == 'yes': v = 2
else: v = 0
hist = classifier.fit(featurevals_train, labels_train,
batch_size=batchsize, epochs=num_epochs, verbose=v)
print('Number of epochs:', num_epochs)
print('Final accuracy on training data:', hist.history['accuracy'][-
1])
# Evaluate on test data
test_acc = classifier.evaluate(featurevals_test, labels_test,
verbose=0)[1]
print('Accuracy on test data:', test_acc)
```

```
/usr/local/lib/python3.12/dist-packages/keras/src/layers/core/
dense.py:93: UserWarning: Do not pass an `input_shape`/`input_dim`
argument to a layer. When using Sequential models, prefer using an
`Input(shape)` object as the first layer in the model instead.
  super().__init__(activity_regularizer=activity_regularizer,
**kwargs)
```

```
Number of epochs: 20
Final accuracy on training data: 0.8302808403968811
Accuracy on test data: 0.8611111044883728
```